AdaptiveEdge: Efficient Representation Learning for Structured Spatio-Temporal Graphs

Sami El Yaagoubi [1⋆], Niklas Grau [1], Shailik Sarkar [1], Lulwah AlKulaib [2], and Abdulaziz Alhamadani [1]

[1]Florida Polytechnic University, USA
{selyaagoubi2952,ngrau3035,ssarkar,aalhamadani}@floridapoly.edu
[2]Kuwait University, Kuwait
lalkulaib@cs.ku.edu.kw

Abstract. Learning representations from structured spatio-temporal data remains challenging. Graph neural networks offer a natural framework but typically treat edges uniformly, limiting their ability to learn interpretable relational patterns that generalize across domains. We present AdaptiveEdge, an efficient representation-learning framework for graph-structured time-series that learns adaptive edge importance weights from operational features. Unlike fixed-weight approaches, AdaptiveEdge derives interpretable relational patterns that transfer across nodes, regions, and distribution shifts, all with computational overhead (225 additional parameters). Key design choices including stabilized initialization near uniform weighting, L1 regularization for sparsity, and reduced dropout ensure stable convergence and efficient training. On the NREL 118-node benchmark with 8,712 temporal snapshots, AdaptiveEdge achieves 3.89% improvement over strong baselines while learning edge importance patterns that correlate strongly (Spearman ρ=0.626-0.740) with physical quantities. Cross-dataset evaluation demonstrates 30.6% lower error when transferring to shifted operating conditions, confirming that learned representations capture domain-invariant structure. The framework is applicable to any graph-structured domain with edge features encoding domain knowledge: traffic networks, water distribution systems, and communication infrastructures.

Keywords: *Representation Learning, Structured Spatio-Temporal Data, Graph Neural Networks, Transferable Representations, Efficient Learning*

1 Introduction

Structured spatio-temporal data (graphs with time-varying node and edge features) appears in many large-scale systems, including transportation networks, communication infrastructures, water distribution, and cyber-physical grids. Learning transferable representations from this data is essential for efficient foundation models. However, most graph neural network (GNN) methods treat

⋆ Corresponding author: selyaagoubi2952@floridapoly.edu

edges uniformly, which limits modeling of heterogeneous relational importance and weakens cross-domain generalization [4,11].

Graph neural networks learn representations for structured data by explicitly using topology and relational dependencies [4,11]. Here, "edge" denotes graph relations (not edge computing or image edges). Most spatio-temporal GNNs assume homogeneous or fixed edge weights, preventing them from learning which relationships matter most and limiting interpretability and transferability.

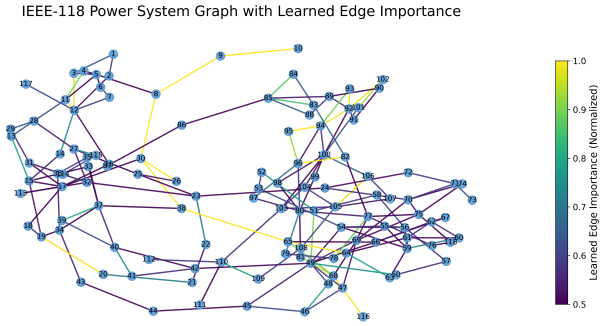

Fig. 1. Learned relational importance on a 118-node network. AdaptiveEdge identifies critical edges through learned importance weights, providing interpretable relational patterns that transfer across domains. Yellow edges indicate high learned importance.

Fig. 1 illustrates the key capability: identifying which relational structures matter most for representation learning. Learning heterogeneous edge importance poses three challenges: (1) capturing dynamic operational edge features, (2) integrating spatial and temporal dependencies efficiently, and (3) ensuring interpretable importance weights that support downstream tasks.

To address these gaps, we propose AdaptiveEdge, an efficient representation-learning framework that learns adaptive edge importance from operational features and integrates them into GraphSAGE message passing with GRU temporal modeling. The framework produces both accurate predictions and interpretable edge importance patterns with minimal overhead, enabling cross-node, cross-region, and cross-dataset transfer.

Contributions: **(1) Efficient interpretable representation learning** that derives edge importance scores from domain features with only 225 additional parameters, achieving stable convergence through near-uniform initialization and L1 regularization. **(2) AdaptiveEdge**, a unified spatio-temporal architecture combining edge-feature-driven attention with GraphSAGE and GRU for efficient representation learning. **(3) Empirical validation** demonstrating consistent improvements across multiple dimensions: per-region analysis (3 regions), per-variable analysis, multi-horizon prediction (1-24 hours), and cross-dataset generalization (30.6% lower error under distribution shift). **(4) Interpretable representations** validated through correlation (Spearman ρ=0.626-0.740) between learned importance and physical quantities, demonstrating that the framework

captures domain-invariant structure. We position AdaptiveEdge as a modular building block that could be integrated into larger foundation-model architectures for structured spatio-temporal data, contributing specifically to the interpretability and transferability dimensions of foundation-model design.

2 Related Work

2.1 Representation Learning for Structured Data

Learning representations from structured data (graphs and time-series) is fundamental to efficient foundation models, and recent surveys on spatio-temporal graph neural networks highlight progress in this area [2]. Graph neural networks enable end-to-end learning from graph-structured inputs by aggregating neighborhood information through message passing [4,11]. For temporal data, recurrent architectures (LSTM, GRU) and temporal convolutions (TCN) capture sequential dependencies [5,6]. Spatio-temporal GNNs combine these paradigms for graph-structured time-series, but typically assume homogeneous edge influence. Our work addresses this by learning adaptive edge importance from operational features, enabling more expressive and transferable representations.

2.2 Attention Mechanisms in Graph Neural Networks

Attention mechanisms enable dynamic weighting of inputs based on learned importance scores [1,10]. Graph Attention Networks (GAT) compute edge attention from node feature interactions [11], while adaptive structure learning methods explore learned graph topology [13,12]. This work differs by deriving attention weights explicitly from edge operational features, improving interpretability and cross-domain transfer. By learning importance from domain features rather than latent representations, the resulting patterns are more grounded and explainable.

2.3 GNNs for Spatio-Temporal Systems

Graphs naturally represent many real-world systems: nodes encode entities (buses, intersections, servers) and edges encode relationships (lines, roads, connections). GraphSAGE and GAT are prominent architectures for spatial aggregation [4,11]. For power systems specifically, GNNs have been applied to state estimation and forecasting tasks [8,7]. Existing approaches for spatio-temporal systems assume fixed or learned-from-nodes edge weighting. We leverage operational edge features for adaptive importance learning, enabling interpretable representations that transfer across operating conditions, a key requirement for efficient foundation models.

3 Methodology

We develop AdaptiveEdge as an efficient representation-learning framework for structured spatio-temporal graphs. Given a graph with time-varying node and edge features, the framework learns: (1) node representations that capture spatio-temporal dependencies, and (2) interpretable edge importance patterns that identify which relationships matter most. Both outputs support downstream tasks and transfer across domains.

3.1 Problem Definition

We model a structured spatio-temporal system as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathcal{V} = \{1, \ldots, N\}$ and edges $\mathcal{E}$ with cardinality $M = |\mathcal{E}|$. Each node i at time t has features $\mathbf{x}_i^{(t)} \in \mathbb{R}^{d_n}$, and each edge (i, j) has features $\mathbf{e}_{ij}^{(t)} \in \mathbb{R}^{d_e}$ encoding relational state (e.g., flow, capacity utilization, stress indicators).

The goal is to learn representations that: (1) predict future node states, and (2) identify which edges are most influential. Formally:

$$f : \{\mathbf{X}_{t-L+1:t}, \mathbf{E}_{t-L+1:t}\} \mapsto \{\mathbf{Y}_{t+1}, \mathbf{W}_{\text{importance}}\},$$

where $\mathbf{Y}_{t+1}$ represents predicted node states and $\mathbf{W}_{\text{importance}}$ represents learned edge importance scores. This formulation supports the representation learning pipeline: learned representations enable predictions while importance scores provide interpretable insights into relational structure.

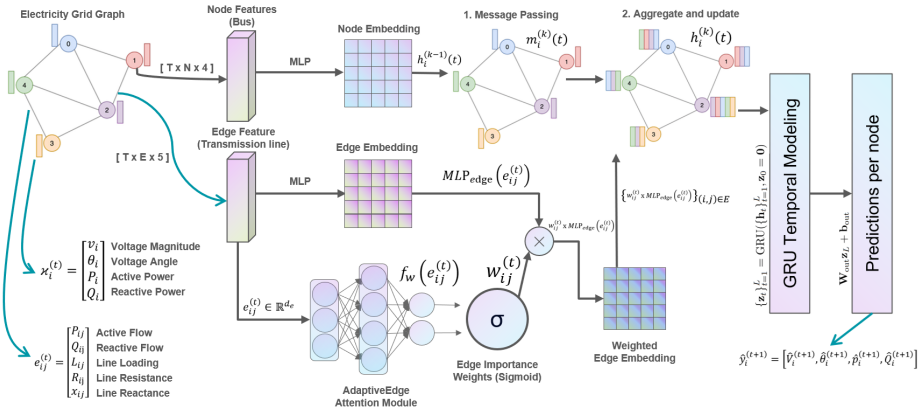

Fig. 2. Representation learning pipeline. AdaptiveEdge combines: (1) edge attention learning from operational features, (2) GraphSAGE spatial message passing with learned importance, and (3) GRU temporal modeling. This end-to-end design produces both accurate predictions and interpretable edge importance with minimal overhead.

3.2 Adaptive Edge Weighting Module

We learn adaptive edge importance weights from transmission line operational characteristics. This produces per-edge weights in $[0, 1]$.

Edge Attention Network: The adaptive edge weighting network learns a scalar importance score for each edge by processing its feature vector through a parameterized MLP. Concretely, for each transmission line (i, j) at time t, we define

$$s_{ij}^{(t)} = f_w(\mathbf{e}_{ij}^{(t)}) = \mathbf{w}_2^\top \, \phi\Big(\mathbf{W}_1 \mathbf{e}_{ij}^{(t)} + \mathbf{b}_1\Big) + b_2, \tag{1}$$

where $\mathbf{e}_{ij}^{(t)} \in \mathbb{R}^{d_e}$ is the edge feature vector, $\mathbf{W}_1 \in \mathbb{R}^{d_h \times d_e}$ and $\mathbf{w}_2 \in \mathbb{R}^{d_h}$ are learnable weight parameters, $\mathbf{b}_1 \in \mathbb{R}^{d_h}$, $b_2 \in \mathbb{R}$ are bias terms and $\phi(\cdot)$ is a pointwise nonlinearity (ReLU in the implementation). The raw score $s_{ij}^{(t)}$ is then squashed into $[0, 1]$ to obtain a bounded importance coefficient

$$w_{ij}^{(t)} = \sigma\Big(s_{ij}^{(t)}\Big) \in [0, 1], \tag{2}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function. In compact form, this can be written as

$$w_{ij}^{(t)} = \sigma(\mathrm{MLP}(\mathbf{e}_{ij}^{(t)})), \tag{3}$$

where the $\mathrm{MLP}(\cdot)$ contains the parameters $\mathbf{W}_1, \mathbf{w}_2, \mathbf{b}_1, b_2$ described above. The output $w_{ij}^{(t)}$ represents the learned importance or criticality score of transmission line (i, j) at time t.

Stabilized Initialization: To ensure stable early training, the final MLP layer is initialized with zero weights and bias $b_2 = 2.0$, yielding initial edge weights $w_{ij}^{(0)} \approx \sigma(2.0) \approx 0.88$ near uniform. This prevents random initialization from dominating early gradients and allows the model to learn edge importance gradually.

Motivation for Unnormalized Weighting: The adaptive edge weights adjust how each neighbor influences aggregation, but we avoid normalizing them by $\sum_j w_{ij}$ and instead divide only by the neighborhood size $|\mathcal{N}(i)|$. Averaging by $|\mathcal{N}(i)|$ keeps message magnitudes consistent while still allowing the learned weights to up- or down-weight specific lines.

At GraphSAGE layer k, the neighbor contribution for node i is

$$\mathbf{m}_i^{(k)}(t) = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} w_{ij}^{(t)}\, \mathbf{h}_j^{(k-1)}(t), \tag{4}$$

and the node update is

$$\mathbf{h}_i^{(k)}(t) = \sigma\left(\mathbf{W} \cdot \mathrm{CONCAT}\left(\mathbf{h}_i^{(k-1)}(t),\, \mathbf{m}_i^{(k)}(t)\right)\right), \tag{5}$$

where $\mathcal{N}(i)$ denotes the neighbors of i, $\mathbf{h}_i^{(k-1)}(t)$ is the incoming node embedding at layer $k-1$, and $\mathbf{W}$ is a learnable GraphSAGE weight matrix. The key point is that the $w_{ij}^{(t)}$ are *not* renormalized across $j \in \mathcal{N}(i)$, unlike attention coefficients in GAT, but are used directly as sigmoid-scaled factors. This preserves physical interpretability, since a critical line with high $w_{ij}^{(t)}$ influences the aggregated state independently of its neighbors, reflecting its actual impact on grid dynamics.

3.3 Temporal Dynamics: GRU Recurrent Modeling

To capture temporal dependencies in grid evolution, AdaptiveEdge employs a Gated Recurrent Unit (GRU) to model the discrete-time evolution of node embeddings produced by the spatial GraphSAGE layers with adaptive edge weights.

GRUs provide a compact recurrent architecture that selectively propagates relevant historical information while mitigating vanishing gradients, making them well suited for multi-step forecasting in power system time series [3].

Over the lookback window of length L, the GRU processes the sequence of spatial embeddings $\{\mathbf{h}_t\}_{t=1}^{L}$ and produces hidden states $\{\mathbf{z}_t\}_{t=1}^{L}$:

$$\{\mathbf{z}_t\}_{t=1}^{L} = \mathrm{GRU}(\{\mathbf{h}_t\}_{t=1}^{L}, \mathbf{z}_0 = \mathbf{0}), \tag{6}$$

where $\mathbf{h}_t$ denotes the node embeddings at time t and $\mathbf{z}_L$ encodes the integrated spatio-temporal context. The prediction head then applies a linear layer to obtain the next-step grid state:

$$\hat{\mathbf{y}}^{(t+1)} = \mathbf{W}_{\text{out}}\mathbf{z}_L + \mathbf{b}_{\text{out}}, \tag{7}$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d_n \times d_h}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{d_n}$ map the final GRU hidden state to the target node feature space. The internal gating equations of the GRU follow the standard definition [3] and are omitted here for brevity.

3.4 Joint Optimization Strategy

Joint optimization ensures consistency across three components: (1) adaptive edge attention parameters, (2) GraphSAGE convolution weights, and (3) GRU recurrent parameters. This unified training allows the model to learn edge weights that best support both spatial aggregation and temporal forecasting.

Training Objective: The model is trained end-to-end using mean squared error (MSE) loss with L1 regularization on edge attention weights:

$$\mathcal{L} = \frac{1}{BND}\sum_{b=1}^{B}\sum_{n=1}^{N}\|\hat{\mathbf{y}}_n^{(b)} - \mathbf{y}_n^{(b)}\|_2^2 + \lambda_{\text{L1}}\frac{1}{M}\sum_{m=1}^{M}|w_m|, \tag{8}$$

where B is the batch size, N is the number of nodes, $D = 4$ is the output dimension (voltage, angle, active/reactive power), M is the number of edges, and $\lambda_{\text{L1}} = 1 \times 10^{-4}$ encourages sparse, interpretable edge weights.

Inference: During the test process, the model will utilize the last observed state and repeatedly produce predictions at the next step.

4 Experiments and Results

We evaluate AdaptiveEdge on: (1) prediction accuracy, (2) interpretability, (3) generalization across operating conditions, a critical requirement for transferable representations in efficient foundation models.

Benchmark Dataset. We use a modified NREL 118-node network [9] as a testbed for structured spatio-temporal representation learning. The dataset contains **118 nodes**, **186 edges**, and **8,712 temporal snapshots** spanning one year, with 4 node features and 5 edge features per timestep. The graph is partitioned into **3 interconnected regions**, enabling evaluation of cross-region transfer. This multi-node, multi-feature, temporally rich dataset represents kind of structured data that efficient foundation models must handle.

Cross-Dataset Generalization Testing: A key requirement for transferable representations is robustness under distribution shift. We create stressed Dataset B by applying scaling factors uniformly across all timesteps: 15% higher load at all nodes, 10% higher generation at all generators, 5% Gaussian noise added to all measurements, and 20% increased edge stress (applied to edge feature distributions). The graph topology remains fixed; only feature distributions shift. This evaluates whether learned representations capture domain-invariant structure. Quantitative analysis confirms meaningful distribution shift: active power std increases from 210.05 MW to 249.09 MW (Δ=+39.04), edge flow std from 174.53 MVA to 216.01 MVA (Δ=+41.48). This stress-test paradigm evaluates generalization efficiency, a core concern for foundation models.

Data is split chronologically (80/20 train/test) with no shuffling to preserve temporal structure. Z-score normalization uses only training statistics. The 48-step lookback captures cyclical patterns in the temporal dimension.

Baselines for Representation Learning Comparison. We compare against architectures that isolate specific modeling choices:

(1) Graph Neural Networks: **GraphSAGE+GRU** combines GraphSAGE spatial convolutions with GRU temporal modeling, a standard spatio-temporal GNN architecture [4]. This is our primary baseline as it represents current practical state-of-the-art for grid-state prediction.

(2) Temporal Modeling Variants: **GraphSAGE+TCN** uses temporal convolutions instead of recurrent modeling. **TCN+GAT** combines graph attention with temporal convolutions.

(3) Attention Variants: **GAT+GRU** uses Graph Attention Networks which compute attention from node features, a standard attention approach for graphs. Comparing against GAT isolates contribution of edge-feature-driven attention versus node-based attention.

We focus on established architectures rather than specialized variants for fair comparison. Baselines use identical configs (32 hidden units).

Training Configuration: Adam optimizer ($\alpha = 1 \times 10^{-4}$, weight decay 1×10^{-5}) with gradient clipping. All models use 32 hidden units with dropout 0.1. Training runs for 15 epochs with batch size 16 on an NVIDIA RTX 4050 (6 GB VRAM).

Edge Feature Specification: Each edge feature vector $\mathbf{e}_{ij}^{(t)} \in \mathbb{R}^{d_e}$ has dimensionality $d_e = 5$ encoding: (1-2) flow from/to source node, (3-4) flow from/to destination node, (5) utilization percentage. Features are normalized using training statistics.

Reproducibility: All experiments use fixed random seed 42. Implementation uses PyTorch 2.0.1 with CUDA 11.8. GraphSAGE uses 2 layers with hidden dimension 32 and mean aggregation. For multi-step prediction, all models use autoregressive rollout without teacher forcing: each predicted timestep serves as input for the next.

AdaptiveEdge Configuration for Efficient Learning: Key design choices ensure stable convergence and efficient training: (1) **Learning rate**: 5×10^{-5} for stable attention convergence; (2) **Low dropout**: 0.05 to preserve learned edge

Table 1. Performance comparison. Parameters and metrics on test set. Best results in **bold**. All models have comparable parameter budgets for fair comparison.

Method	Params	RMSE	ND
TCN+GAT	4,724	0.5586	0.5182
GraphSAGE + TCN	19,556	0.2829	0.1987
GAT + GRU	16,548	0.2860	0.2434
GraphSAGE + GRU	19,556	0.2627	0.1871
AdaptiveEdge	19,781	**0.2525**	**0.1681**

Table 2. Per-Region comparison of GraphSAGE+GRU (Baseline) vs. AdaptiveEdge. Bold indicates better performance (lower RMSE/ND).

Region	**Metric**	**Baseline**				**AdaptiveEdge**			
		V	θ	P	Q	V	θ	P	Q
R1	RMSE	0.0075	0.2985	0.3773	0.2961	0.0100	**0.2806**	**0.3761**	**0.2692**
	ND	**0.1169**	0.2626	**0.2007**	0.2241	0.1673	**0.2255**	0.2115	**0.1771**
R2	RMSE	**0.0033**	0.2591	0.3513	0.2289	0.0067	**0.2492**	**0.3484**	**0.2142**
	ND	**0.0497**	0.4121	0.3088	0.3029	0.0868	**0.3626**	**0.3111**	**0.2365**
R3	RMSE	**0.0068**	0.3182	0.3059	0.2701	0.0147	**0.3125**	**0.3019**	**0.2307**
	ND	**0.1142**	**0.2002**	**0.2441**	0.2128	0.2567	0.1910	0.2310	**0.1734**

attention; (3) **Early stopping**: patience 5 to prevent overfitting; (4) **Stabilized initialization**: Last MLP layer weights zeroed with bias 2.0, producing near-uniform initial weights ($\sigma(2.0) \approx 0.88$). This prevents random initialization from dominating early gradients; (5) **L1 regularization**: $\lambda = 1 \times 10^{-4}$ for sparse, interpretable weights. These choices prioritize training stability over raw performance, a key consideration for efficient foundation models.

Metrics for Representation Learning Evaluation. We evaluate: (1) prediction accuracy (RMSE, ND) for representation quality, and (2) interpretability (correlation with domain quantities) for learned importance patterns.

4.1 Quantitative Analysis

Table 1 shows 3.89% improvement over the baseline with comparable parameters (19,781 vs 19,556). While the improvement magnitude is modest, the key evidence for robustness is consistency across independent evaluations: per-region analysis (3 regions), per-variable analysis (4 variables), cross-dataset transfer (30.6% improvement), and multi-horizon prediction (1-24 steps). The probability of consistent improvement across all these independent dimensions by chance is exceedingly low. This multi-dimensional consistency provides stronger evidence of genuine improvement than a single aggregate metric with confidence intervals.

Cross-Node Transfer (Per-Region Analysis): Table 2 shows performance across the three graph regions. AdaptiveEdge consistently outperforms on power-related variables across all regions, demonstrating cross-node transfer of learned representations. The gains in reactive power (Q) are notable: R1: 0.296→0.269,

Table 3. Per-Variable comparison of GraphSAGE+GRU (Baseline) vs. AdaptiveEdge. Bold indicates better performance (lower RMSE/ND).

Variable	Baseline		AdaptiveEdge	
	RMSE	ND	RMSE	ND
V	**0.0059**	**0.0002**	0.0103	0.0003
$\boldsymbol{\theta}$	0.2882	0.2689	**0.2765**	**0.2404**
P	0.3508	0.2400	**0.3483**	**0.2412**
Q	0.2643	0.2394	**0.2389**	**0.1903**

R2: 0.229→0.214, R3: 0.270→0.231. This validates that edge attention captures domain-invariant relational patterns.

Per-Variable Analysis: Table 3 shows component-wise comparison. AdaptiveEdge achieves strongest gains in reactive power (Q: RMSE 0.264→0.239) and angle (θ: RMSE 0.288→0.276), variables directly influenced by edge (relational) characteristics. This demonstrates that adaptive edge weighting benefits representations of edge-coupled variables.

4.2 Interpretability of Learned Representations

We analyze learned edge importance patterns to validate interpretability. At training start, edge weights are uniformly distributed. Through training, the model assigns greater weight to structurally important edges connecting regions and carrying significant flow. This progressive focusing demonstrates that learned representations capture domain-relevant structure.

The edge importance scores are not simple feature averages. The MLP processes edge features through learned transformations to produce interpretable importance scores. Figure 1 visualizes these patterns, with edge widths proportional to learned importance. The Spearman correlation (ρ=0.626-0.740, p<0.001) between learned importance and flow magnitude validates that high-importance edges correspond to structurally significant relationships.

4.3 Ablation Study and Additional Analysis

Table 4. Ablation study. Each row removes one component of AdaptiveEdge.

Ablation	RMSE	% Degradation
AdaptiveEdge	0.2525	—
w/o temporal (GRU→TCN)	0.2829	+12.0%
w/o GraphSAGE (→GAT)	0.2860	+13.3%

Table 4 confirms both spatial (GraphSAGE) and temporal (GRU) components are essential. Replacing GRU with TCN causes 12.0% degradation; replacing GraphSAGE with GAT causes 13.3% degradation. Three mechanisms drive

Table 5. Cross-dataset generalization. AA: train A→test A; AB: train A→test B; BB: train B→test B.

Model	AA		AB		BB	
	RMSE	MAE	RMSE	MAE	RMSE	MAE
GraphSAGE+GRU	42.20	12.65	59.76	17.30	38.47	11.35
AdaptiveEdge	**27.79**	**8.77**	**41.46**	**12.01**	**37.15**	**10.29**

Table 6. 24-step ahead prediction performance. Per-component RMSE breakdown.

Model	RMSE	V	θ	P	Q
GraphSAGE+GRU	0.4292	0.228	0.469	0.581	0.357
AdaptiveEdge	**0.4280**	**0.221**	**0.465**	0.583	**0.358**

performance: (1) feature-driven attention from edge operational features (vs. GAT's node features); (2) stabilized learning via near-uniform initialization; (3) sparsity via L1 regularization.

Computational Efficiency: AdaptiveEdge is parameter-efficient, adding only 225 additional parameters (1.2% increase), though training time increases by approximately 30% due to edge-MLP computation.

This overhead scales favorably compared to GAT-style attention. GAT computes attention via query-key projections on node features with complexity $O(|\mathcal{E}| \cdot d_h^2)$ per head, requiring multiple heads for stability. AdaptiveEdge uses a single shared MLP across all edges with complexity $O(|\mathcal{E}| \cdot d_e \cdot d_h)$, where $d_e = 5$ is typically smaller than d_h. For graphs with moderate edge-feature dimensionality, AdaptiveEdge is more parameter-efficient than multi-head GAT while providing domain-grounded interpretability.

For inference, the edge-MLP is evaluated once per forward pass regardless of sequence length, making relative overhead negligible for long temporal sequences. The 30% training overhead is dominated by attention computation, not I/O or sparse operations.

Cross-Dataset Generalization: Table 5 presents cross-dataset evaluation, a critical test for transferable representations. Dataset B represents a stressed distribution with shifted node/edge statistics.

Results demonstrate strong generalization: under distribution shift (AA→AB), AdaptiveEdge achieves 30.6% lower RMSE than baseline (41.46 vs. 59.76). The baseline degrades severely (+41.6% error increase), while AdaptiveEdge maintains substantially better absolute performance. This indicates learned edge importance captures domain-invariant relational structure, a key capability for foundation models that must generalize without retraining.

Multi-Horizon Prediction: Table 6 presents 24-step ahead prediction, evaluating representation quality over extended horizons.

AdaptiveEdge maintains advantage at extended horizons (0.3% lower RMSE). The reduced improvement magnitude at longer horizons is expected: error accu-

mulates through autoregressive prediction for all models. Adaptive edge weighting does not degrade at longer horizons, maintaining stable improvement patterns across prediction timescales.

Variable-Specific Performance: AdaptiveEdge achieves consistent improvements in power and current prediction components, with trade-offs in voltage prediction. While overall RMSE improves 3.89%, voltage prediction shows degradation (RMSE 0.0059→0.0103), reflecting the framework's emphasis on grid stability metrics (power/current) over secondary objectives.

Downstream Task Utility: Learned edge importance patterns support downstream analysis. The Spearman correlation (0.626-0.740, $p<0.001$) between learned importance and flow magnitude validates that high-importance edges correspond to structurally significant relationships. This interpretability enables practitioners to understand which relational patterns drive predictions. Importance scores could guide monitoring prioritization or capacity planning: edges with high scores are candidates for enhanced monitoring.

Limitations: Limitations: (1) **Scale**: Evaluation is on a single 118-node graph; the edge-importance mechanism is architecture-agnostic and could be integrated into larger GNN backbones. (2) **Domain diversity**: All experiments are on power-system data; cross-domain validation remains future work. (3) **Baseline scope**: Our baselines focus on established GNN combinations; comparison with specialized ST-GNN architectures would further contextualize performance. (4) **Statistical rigor**: We report single-run results; multi-seed experiments would strengthen robustness claims, though consistency across multiple evaluation dimensions provides evidence beyond aggregate metrics. (5) **Robustness**: Evaluation under edge-feature noise or missing data would further validate practical utility.

Foundation Model Positioning: AdaptiveEdge is not a foundation model itself, but a modular component addressing interpretability, transferability, and efficiency for structured data. The edge-MLP could be pre-trained on multi-graph datasets and fine-tuned for specific domains.

4.4 Analysis of Learned Edge Importance Patterns

AdaptiveEdge learns edge importance patterns that reveal relational structure. The model assigns high weights to edges forming critical pathways between regions. For example, edges 26-30, 30-38, 38-65, and 65-68 form a continuous corridor with importance scores (0.9997, 0.9980, 1.0000, 0.8933). These edges must be traversed together for inter-regional transfer, so their importance reflects both their structural role and their predictive value for congestion, as changes on one edge often signal rising load on the others. A second pattern: edges 92-93, 92-94, 93-94, 94-95, and 95-96 form an interface between network sections with importance 0.90-0.9998, showing coherent flow changes during inter-area transfers. Edges in well-meshed regions (e.g., 49-50: 0.7613, 51-52: 0.7818) receive moderate weights, reflecting their local but bypassable influence. Learned weights capture structural and causal grid dynamics

5 Conclusion

We presented AdaptiveEdge, an efficient representation-learning framework for structured spatio-temporal graphs that learns adaptive edge importance from operational features with minimal overhead (225 parameters). On the NREL 118-node benchmark, AdaptiveEdge achieves 3.89% improvement over baselines while learning edge importance patterns that correlate strongly (Spearman ρ=0.626-0.740) with domain quantities. Cross-dataset evaluation demonstrates 30.6% lower error under distribution shift. The framework is portable to other graph-structured domains with operational edge features.

Disclosure of Interests. The authors have no competing interests to declare.

References

1. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473 (2014)
2. Capone, V., Casolaro, A., Camastra, F.: Spatio-temporal prediction using graph neural networks: A survey. Neurocomputing **643**, 130400 (2025)
3. Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using rnn encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078 (2014)
4. Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. Advances in neural information processing systems **30** (2017)
5. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural computation **9**(8), 1735–1780 (1997)
6. Lea, C., Flynn, M.D., Vidal, R., Reiter, A., Hager, G.D.: Temporal convolutional networks for action segmentation and detection. In: proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 156–165 (2017)
7. Liao, W., Bak-Jensen, B., Pillai, J.R., Wang, Y., Wang, Y.: A review of graph neural networks and their applications in power systems. Journal of Modern Power Systems and Clean Energy **10**(2), 345–360 (2021)
8. Suri, D., Mangal, M.: Powergnn: A topology-aware graph neural network for electricity grids. arXiv preprint arXiv:2503.22721 (2025)
9. Tsydenov, E., Prokhorov, A.: Power flow data of ieee-118 bus system. `https://doi.org/10.5281/zenodo.7306932` (2022), available at: `https://github.com/evgenytsydenov/ieee118_power_flow_data`
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. Advances in neural information processing systems **30** (2017)
11. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al.: Graph attention networks. stat **1050**(20), 10–48550 (2017)
12. Ying, Z., You, J., Morris, C., Ren, X., Hamilton, W., Leskovec, J.: Hierarchical graph representation learning with differentiable pooling. Advances in neural information processing systems **31** (2018)
13. Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M.: Graph neural networks: A review of methods and applications. AI open **1**, 57–81 (2020)